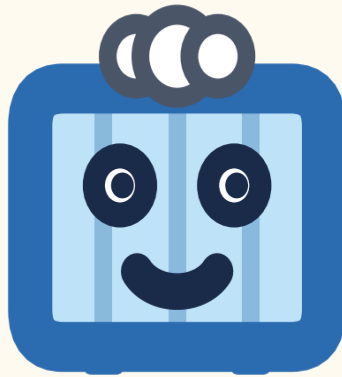


GOOGLE CLOUD

How GCP's core services fit together — the illustrated field guide



with Cargo, your guide

Part of the Interview-Preparation illustrated series

Table of Contents

1. Resource Hierarchy
2. GKE: Standard vs. Autopilot
3. BigQuery: Storage and Compute, Decoupled
4. Spanner: Global AND Strongly Consistent
5. Pub/Sub: One Service, Two Jobs
6. Cloud Run: Any Container, No Cluster
7. Service Accounts: Identity for Workloads
8. Cheat Sheet

Many GCP concepts map directly onto AWS equivalents -- this guide focuses on the handful of ideas that are genuinely distinctive to Google Cloud's design.

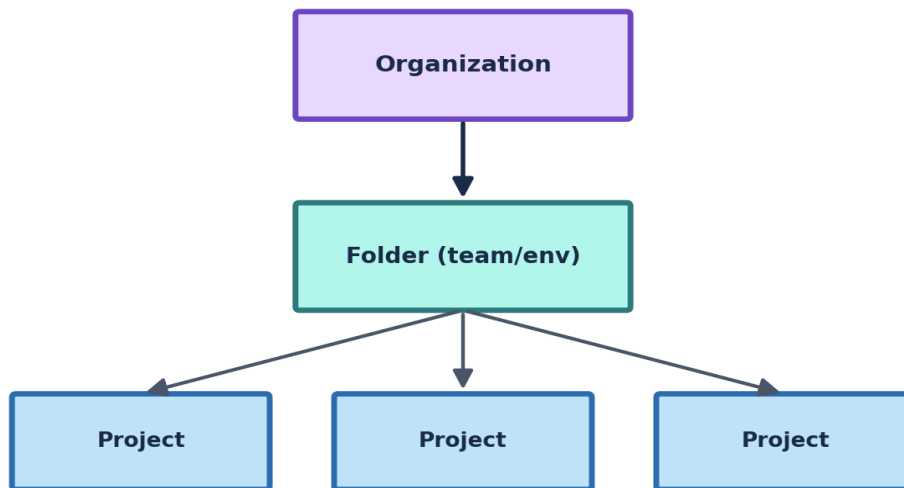
CHAPTER 1

Resource Hierarchy

GCP organizes everything under a strict hierarchy: an Organization at the root, optional Folders beneath it (typically mirroring teams or environments), and Projects as the actual unit that owns resources, billing, and API access.

IAM permissions set at any level flow downward automatically. Grant someone a role at the Folder level, and they inherit it across every Project inside that folder -- without needing to configure each Project individually.

Resource Hierarchy



IAM permissions set higher up are inherited by everything beneath them.

IN SHORT: Permissions inherit downward through Organization -> Folder -> Project. Set access at the highest sensible level, not project by project.

CHAPTER 2

GKE: Standard vs. Autopilot

Standard mode gives you a real Kubernetes cluster where you provision and manage the underlying node VMs yourself -- full control over machine types, but also full responsibility for capacity planning and node-level patching.

Autopilot mode hands node management entirely to Google. You describe what your Pods need (CPU, memory), and Google provisions and manages the underlying infrastructure transparently -- billed per Pod resource consumed, not per provisioned node.

GKE: Standard vs. Autopilot

Standard

You manage nodes

Full control over
machine types,
you pay per node

Autopilot

Google manages nodes

You specify Pod needs,
Google handles the rest,
pay per Pod resource



IN SHORT: Standard = you manage nodes, more control. Autopilot = Google manages nodes, less operational overhead. Same Kubernetes API either way.

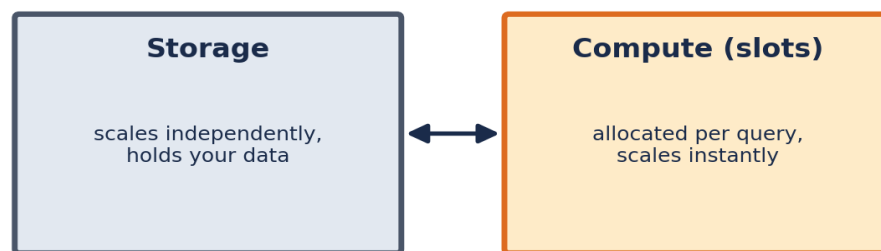
CHAPTER 3

BigQuery: Storage and Compute, Decoupled

Many traditional data warehouses tie storage and compute together in one fixed cluster -- scale one, and you're forced to scale the other too, whether you need it or not.

BigQuery separates them entirely. Your data sits in storage that scales independently, and query compute ("slots") is allocated fresh for each query -- meaning many different queries can run concurrently against the same data without competing for a shared, fixed-size cluster the way older architectures require.

BigQuery: Storage and Compute, Decoupled



Many independent queries can run concurrently against the same data without contending for a shared, fixed compute cluster.

IN SHORT: Storage scales on its own. Compute scales per query, instantly. That's what makes BigQuery feel serverless even for genuinely massive datasets.

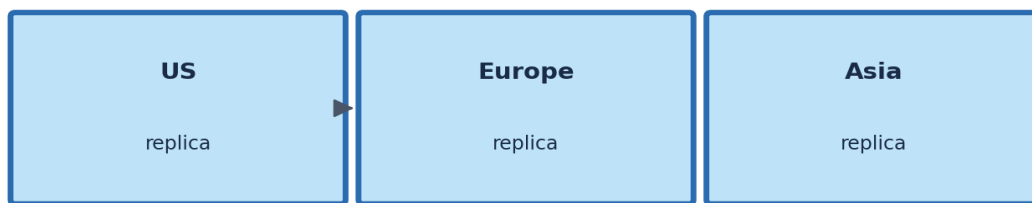
CHAPTER 4

Spanner: Global AND Strongly Consistent

The CAP theorem says a distributed system generally can't have strong consistency and high availability across a network partition at the same time. Spanner is one of the few real systems that gets remarkably close to having both, globally.

It does this with TrueTime -- an API backed by atomic clocks and GPS receivers in Google's data centers, providing a tightly-bounded, known uncertainty window on the current time. Spanner uses that bounded uncertainty to correctly order transactions across continents without the heavy coordination overhead a purely software-clock-based system would need.

Spanner: Global AND Strongly Consistent



TrueTime (atomic clocks + GPS) lets Spanner order transactions correctly across continents -- without sacrificing consistency for availability.



IN SHORT: Most distributed databases pick consistency or low-latency global availability. Spanner's specialized hardware-backed clock synchronization lets it offer both.

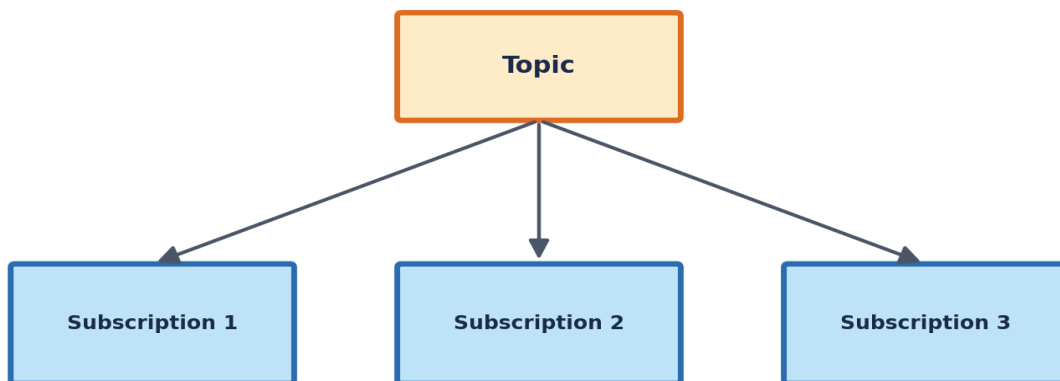
CHAPTER 5

Pub/Sub: One Service, Two Jobs

AWS splits this into two separate services: SNS for fan-out pub/sub, SQS for reliable point-to-point queuing. GCP's Pub/Sub combines both roles into one service.

Publish a message to a Topic, and every current Subscription attached to it receives its own full, independent copy -- giving you fan-out (like SNS) with each individual subscription also behaving like a reliable, acknowledgment-based queue (like SQS).

Pub/Sub: One Service, Two Jobs



Every subscription gets its own full copy of every message -- fan-out (SNS) and reliable queuing (SQS) in one service.

IN SHORT: One topic, many independent subscriptions, each getting every message -- fan-out and reliable queuing, unified into a single service.

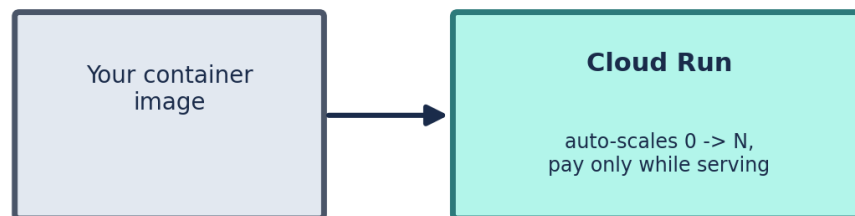
CHAPTER 6

Cloud Run: Any Container, No Cluster

Cloud Run takes any container image -- not just a single function following a specific handler signature, but a genuinely arbitrary container with any language or framework -- and runs it with fully serverless scaling.

It scales from zero (no traffic, no cost) up to however many instances are needed to handle load, and back down again, without you ever provisioning or managing a cluster.

Cloud Run: Any Container, No Cluster



IN SHORT: The flexibility of a real container, the operational simplicity of serverless. No cluster to think about, no idle cost.

CHAPTER 7

Service Accounts: Identity for Workloads

This is GCP's direct equivalent to AWS's IAM Roles. A Service Account represents a non-human identity, attached to a workload -- a VM, a Cloud Run service, a GKE Pod -- rather than a person.

The workload authenticates as that Service Account automatically, with no long-lived credentials embedded anywhere. Grant the Service Account exactly the permissions the workload actually needs, nothing more.

Service Accounts: Identity for Workloads



Same principle as AWS IAM Roles: attach an identity to the workload, never embed long-lived keys.

IN SHORT: Same principle as AWS: attach an identity to the workload itself, and let the platform handle credential delivery -- never embed long-term keys.

CHAPTER 8

Cheat Sheet

The core services, grouped by category.

GCP CHEAT SHEET



Compute

Compute Engine - VMs

Cloud Run - serverless containers

Cloud Functions - serverless functions

GKE - managed Kubernetes

Storage & Database

Cloud Storage - object storage

Cloud SQL - managed relational DB

Firestore - NoSQL document DB

Spanner - global relational DB

Data & Analytics

BigQuery - serverless data warehouse

Dataflow - stream/batch pipelines

Pub/Sub - messaging

Networking

VPC - virtual network

Cloud Load Balancing - global LB

Cloud DNS - DNS hosting

IAM

Organization > Folder > Project

Service Accounts - workload identity